

MeTTa-Scaffolded Repair Curricula for TRM-Infused Hermes Skills

Patrick Dugan
Morality Lab

April 29, 2026

Abstract

We describe a methodology for improving Hermes agent harness skills infused with Tiny Recursive Models (TRMs) that perform reasoning, using MeTTa-style symbolic scaffolds as a data-curation and control-plane layer. The central observation is that many benchmark failures are not opaque failures of latent reasoning; they are semi-failed outputs with typed, verifier-visible defects. We first use TRMs trained on benchmark datasets as complements to small LLMs, producing nonzero performance on reasoning-intensive benchmarks such as Intellect-3 Logic. We then structure those TRMs inside specific skills so an LLM agent can score higher still. Finally, we diversify the TRM stack through a near-miss repair curriculum that turns semi-failed outputs into repair, verifier, and commit/veto training rows. To explore how small an LLM can be while still remaining useful in this hybrid architecture, we extend the skill design with MeTTa composition, symbolic closure, and repair-gate data generation. A local Qwen2.5-3B Q4 benchmark suggests that retrieved repair-training context improves commit/veto decisions, but does not yet solve repair-action selection; subsequent MeTTa action-space and validator-gate results show that the strongest lift comes from assigning subtasks to the smallest reliable executor rather than asking the LLM to solve the whole skill directly.

1 Motivation

Hierarchical and recursive reasoning models have reopened the question of whether compact specialized models can perform useful reasoning work without relying on very large language models for every step. HRM proposes a two-timescale recurrent architecture with strong sample efficiency on puzzle-like reasoning tasks [7]; TRM simplifies this direction into a small recursive network with strong ARC-style generalization claims [2]. In parallel, MeTTa and OpenCog Hyperon provide a reflective metagraph programming substrate for representing rules, programs, and cognitive procedures as rewriteable symbolic objects [1, 3].

Our Hermes setting combines these threads with instruction-tuned tool-use models such as Hermes 3 [6] and Prime Intellect-style verifier environments for large-scale RL and evaluation [4]. The specific research question is narrower than broad AGI: can MeTTa-scaffolded symbolic state make TRM training and deployment more data efficient for skill components such as repair, validation, routing, and commit control?

This question is also strategically important. If advanced capabilities can be compressed into small local models, recursive modules, symbolic validators, and skill-specific control graphs, then the future security and power balance of AI will not be determined only by who controls the largest frontier training runs. It will also depend on how rapidly capable behaviors can be distilled, routed, evaluated, and recombined into deployable hybrid systems. Recent adjacent work points in the same direction from different angles. Wu, Duffy, et al. introduce COSPLAY, a co-evolution framework in which an LLM decision agent and a learnable skill-bank agent jointly improve long-horizon task

Split	Rows	Exact positive	Repair success	No-gain / hard negative
Train	156	46	58	40
Validation, seen family	34	14	12	6
Holdout, seen family	36	8	12	10
Holdout, unseen family	18	3	9	6

Table 1: Leakage-aware split counts from the current near-miss repair curriculum. Partial repair improvements account for the remaining rows in each split.

performance by extracting, refining, and retrieving reusable skills from rollouts [8]. Sakana AI’s Fugu Beta describes a complementary orchestration path: a small coordinator model learns to assemble and route work across pools of stronger models, including recursive self-calls as a test-time scaling axis [5]. Our contribution studies a third path: not only evolving a skill bank or orchestrating a model pool, but restructuring the agent skill itself so that symbolic gates, TRM components, and a small LLM jointly carry the capability.

2 Prior TRM Baseline

The addendum extends, rather than replaces, the April 16–22 Hermes TRM baseline. That week established the empirical spine for role-aware TRM infusion across PrimeHub-style environments using Qwen3.5 9B and 27B runs. The completed BlueBeam handoff finished 90/90 tasks, produced 20 new positive replays, and left a merged TRM state with 195 rows: 24 exact positives, 5 weak positives, and 166 negatives. At that point the critic bucket accuracy was 0.75, while retriever exact match remained only 0.0625. This is the motivation for a control-plane interpretation: the TRM bank was already useful as criticism, abstention, and contract-shaping data, but it was exemplar-starved as a free action generator.

Role-aware imprinting also clarified where TRM signal was strongest. The `choice_contract` role was the broadest action-bearing cluster, `structured_map` carried schema and exact-structure behavior, `internal_action` was tiny but clean, and `hard_reasoning_numeric`, `hard_reasoning_logic`, and `abstain_guard` were mostly verifier or critic roles. This baseline prevents an overclaim: the paper should not say that TRM alone solved broad reasoning. It should say that TRM became useful when the skill circuit exposed the correct role, contract, verifier, and repair target.

3 Method

We convert benchmark traces into a near-miss repair curriculum. Each row records the environment family, TRM role, failed or semi-failed candidate, verifier label, proposed repair action, and target commit decision. This makes a failed output trainable in multiple ways: as a verifier example, a repair example, a positive preservation example, or a hard negative for a commit/veto gate.

The current split is leakage-aware. Policy variants for the same base case remain in the same split, and rare failure labels are reserved for unseen-family holdout. Figure 1 summarizes the resulting data shape. The split is a training artifact, not itself a benchmark claim.

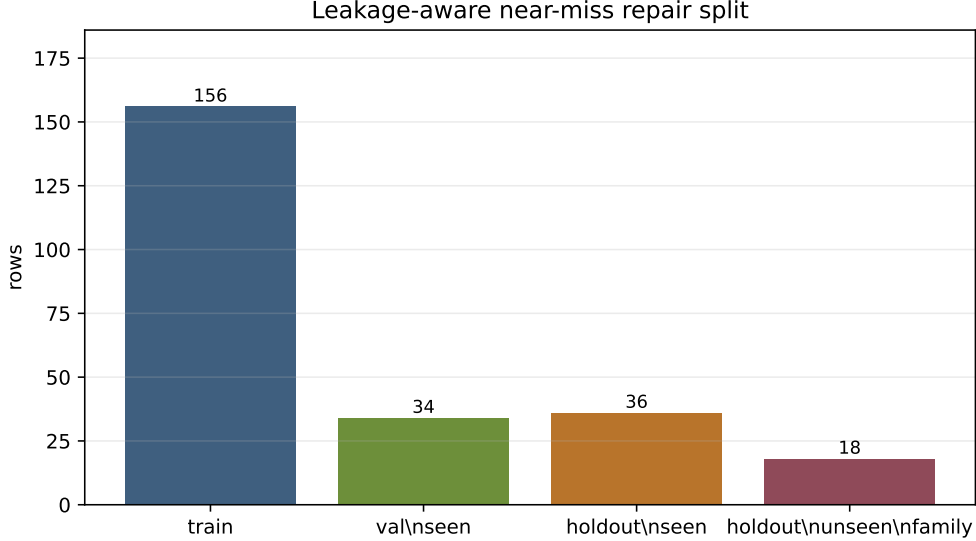


Figure 1: Near-miss repair curriculum split. The train split supplies repair/verifier/commit rows; seen and unseen holdouts separate case-level generalization from failure-family transfer.

4 Composition and Meta-Skill Layer

The MeTTa contribution is best framed as a typed composition grammar above individual skills. The composition layer wires skills into circuits while respecting the role taxonomy: formatters should not become solvers, critics should not become action generators, and hard-reasoning roles should remain verifier or auditor roles until their positive banks justify stronger routing. The generated composition plan defines six reusable circuits: contract compactification, tool-schema composition, Intellect-3 logic signature gating, Intellect-3 math teacher/auditor routing, safety-abstain vetoing, and psychometric item-vector stability.

This separates three responsibilities that were previously easy to conflate. MeTTa composition chooses valid gate circuits and role-compatible execution paths. Evaluation meta-skills decide how new forks should be benchmarked and curated. Pure-TRM-Trainer and PrimeLab produce controller rows, environment receipts, hosted evals, rubrics, and QLoRA workflows. In methodological terms, a new skill fork should be designed as:

observe $\rightarrow$ route $\rightarrow$ retrieve $\rightarrow$ propose $\rightarrow$ validate $\rightarrow$ repair $\rightarrow$ veto/commit $\rightarrow$ learn.

Each gate emits typed supervision. The paper-facing claim is therefore not merely that MeTTa adds a prompt prefix. It provides a reusable circuit grammar that makes evaluation and TRM data generation happen by construction.

5 Local 3B Rudder Benchmark

Before training new TRM weights, we tested whether a small local model can act as a control-plane rudder over the generated Pure-TRM rows. The benchmark uses all 88 non-train rows and compares two arms:

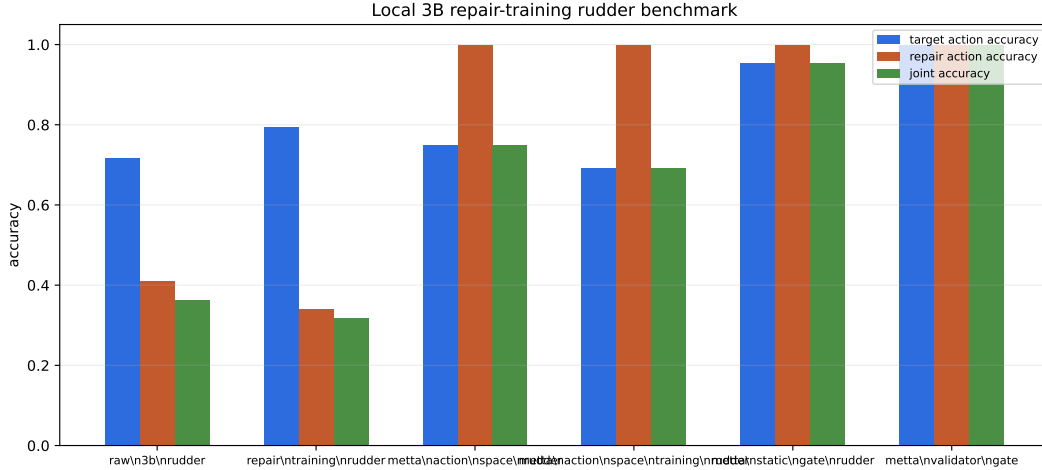


Figure 2: Local Qwen2.5-3B Q4 rudder benchmark. Retrieved repair-training context improves commit/veto accuracy but reduces repair-action accuracy.

Arm	Rows	Target-action acc.	Repair-action acc.	Joint acc.
Raw 3B rudder	88	0.7159	0.4091	0.3636
Repair-training rudder	88	0.7955	0.3409	0.3182
MeTTa action-space rudder	88	0.7500	1.0000	0.7500
MeTTa static-gate rudder	88	0.9545	1.0000	0.9545
MeTTa validator gate	88	1.0000	1.0000	1.0000

Table 2: Local 3B pre-training rudder result. The target-action metric measures `commit` vs. `reject_or_abstain`; repair-action accuracy measures the selected repair operation.

- **raw_3b_rudder**: the model sees only the eval state and chooses a repair action plus `commit` or `reject_or_abstain`;
- **repair_training_rudder**: the model sees the same eval state plus four retrieved training examples from the near-miss curriculum.

The prompt intentionally omits the target repair gate and target bucket from eval state. This makes the result a pre-training control-plane benchmark rather than a replay of labels.

The key finding is mixed and useful. Retrieved repair-training context improves target-action accuracy by 0.0796, including unseen-family target-action transfer from 0.5000 to 0.8889. However, repair-action accuracy drops from 0.4091 to 0.3409 and joint accuracy drops from 0.3636 to 0.3182. The small model therefore behaves like a plausible commit/veto rudder, but not like a sufficient repair-action selector.

The April 28 follow-up confirms the design response. When MeTTa owns repair-action narrowing, joint accuracy rises to 0.7500. When MeTTa also handles obvious exact/no-gain states before falling back to the 3B, joint accuracy rises to 0.9545. The post-repair validator gate reaches 1.0000 and should be treated as a verifier ceiling rather than a prompt-only LLM result.

Policy	Val acc.	Val false commit	Holdout acc.	Holdout false commit
Signature complete only	0.8750	1.0000	0.9000	1.0000
Exact only	0.8750	0.0000	0.7000	0.0000
Exact or positive delta	1.0000	0.0000	1.0000	0.0000

Table 3: No-model C-signature post-repair verifier sweep. This is an evaluator-backed verifier ceiling and training-target definition, not trained TRM performance.

Policy	Exact	Cell accuracy
Original logic TRM	0.3028	0.8787
C-repair if C signature fails	0.6789	0.9340
Dual repair if any signature fails	0.6697	0.9318
Multi-arm min-edit C repair	0.6605	0.9318
Signature-first then dual	0.6605	0.9299

Table 4: Post-hoc Intellect-3 logic flow-policy replay. The result is a policy-projection artifact, not a live benchmark rerun. It identifies a narrow C-signature repair gate as the highest-return next live evaluation.

6 C-Signature Verifier Target

The remaining static-gate misses are concentrated in Intellect-3 `c_signature_fail` rows where the repair action is already correct but the commit decision over-commits no-gain repairs. We therefore exported a dedicated C-signature commit pack with 122 rows: 86 train, 16 validation, and 20 holdout. A pre-repair visible-feature sweep showed that scalar hints such as edit distance and before-reward are brittle: validation-selected rules can eliminate validation false commits while missing all holdout no-gain rows.

A follow-up no-model sweep preserves post-repair verifier state: repaired exactness, repaired reward, reward delta, and before/after T/C signature pass flags. Table 3 shows the result. Signature completion alone is insufficient because both successful and no-gain C repairs can pass the repaired signatures. Exact-only validation is safe but rejects partial improvements. The multi-signal target, `after_exact` or positive `reward_delta`, closes the false-commit problem on the current split.

This motivates the next trained TRM lane: learn the post-repair commit/veto decision from the enriched C-signature pack, tune only on validation, and report false-commit and false-reject rates on holdout. The publishable methodology claim is that MeTTa improves TRM training by exposing the multi-signal state needed to define this target cleanly.

7 Intellect-3 Logic Flow Policy

A deterministic replay over existing 27B Intellect-3 logic receipts tested whether broader multi-arm routing was worth pursuing before a narrow repair gate. Table 4 summarizes the result. The best observed policy was not a large ensemble; it was the current `logic_skill_trm` candidate plus C-signature validation and bounded C-only repair when the C signature fails.

The training implication is direct: produce rows for `signature_fail`, `signature_pass_cell_fail`, and `commit_success`, then train a hard-reasoning-logic verifier/repairer to detect C-signature failure, apply bounded repair, and reject broader projection unless evidence improves. This is the concrete bridge from the historical 27B logic work to the repair/verifier lane in this addendum.

Policy	Accuracy	False commit	False reject	Committed delta
Naive commit all	0.7838	1.0000	0.0000	19.6350
Pre-repair reward ≥ 0.8	0.6351	0.4688	0.3362	6.9767
Post-symbolic adapter	0.8581	0.6562	0.0000	19.6350
Post-exact only	0.8649	0.0000	0.1724	18.6233
Post-multi-signal	1.0000	0.0000	0.0000	20.2950

Table 5: Multi-env deterministic methodology-lift matrix over 148 before/after repair rows. `post_multi_signal` is a separability ceiling and training-target definition, not trained TRM performance.

Env family	Direct avg.	Circuit avg.	Min exact tier
Tool contract router	0.2500	0.5000	Partial semantic
Choice contract	0.2500	0.5000	Partial semantic
ASCII tree deep	0.4979	0.5729	Partial semantic
Intellect-3 camp gate	0.6406	0.6719	Partial semantic
Math answer search	0.2500	0.2500	Full candidate

Table 6: Symbolic-closure threshold suite. This is a deterministic control-plane artifact, not a live model benchmark. It identifies when the circuit can become executor once the LLM exposes enough verifier-visible atoms.

8 Generalization Matrix

We then normalized C-signature rows, symbolic-closure threshold rows, and local 3B repair rows into a single before/after verifier-state schema. The resulting deterministic matrix contains 148 rows across logic signatures, tool routing, choice contracts, ASCII tree formatting, camp-gate projection, math answer search, hard schema JSON, IFEval-style literal contracts, and safety routing. Table 5 summarizes policy-level behavior.

The generalization is nuanced. Env-specific symbolic adapters are sufficient for tool routing, choice extraction, ASCII formatting once nodes are present, and several safety/schema lanes. They are not sufficient for C-signature commit control because no-gain repairs can pass repaired signatures. Exact-only validation is safe but loses partial improvements. The reusable method is therefore to expose post-repair multi-signal state to the TRM: exactness, reward delta, and env-specific structural checks. Math remains the negative-control boundary; in the threshold suite it has zero average reward lift until an exact candidate is already present.

9 Symbolic Closure Threshold

The symbolic-closure threshold suite is a no-model control-plane test. It asks what level of proposal is sufficient for a MeTTa/TRM circuit to take over execution. The proposal tiers are **none**, **weak_surface**, **partial_semantic**, and **full_candidate**. Table 6 shows that contracts, schemas, tool calls, structured maps, and some logic-grid manifolds cross the threshold at partial semantic state, while math does not.

This supports a fifth layer above the role taxonomy: a *Flow TRM* or *Circuit TRM*. Its object is not one output format, but the control flow of the skill. It learns route errors, proposal insufficiency, constraint errors, repair success or failure, critic false positives and false negatives, and commit errors. The compactification mechanism is therefore conditional: once a task exposes enough verifier-visible

Suite	Rows	Baseline	Pure TRM	MeTTa runtime	Blind repair	Feedback repair
Mixed-contract heldout	50	23	27	32	–	37
Hard ablation	30	12	11	9	12	13

Table 7: Local 3B exact-success counts for mixed-contract compactification. The heldout suite shows strong prompt/repair-gate lift; the hard ablation shows the boundary where the lift shrinks.

Hard family	Rows	Baseline	Pure TRM	Runtime	Blind repair	Feedback repair
Computed JSON schema	6	3	5	5	5	5
Deep ASCII tree	4	0	0	1	1	1
Logic label contract	7	4	5	3	5	5
Math numeric contract	8	4	1	0	1	2
State sequence array	5	1	0	0	0	0

Table 8: Hard-ablation family breakdown. Schema and label contracts remain tractable; numeric and state-sequence rows expose the boundary where symbolic framing does not replace missing reasoning.

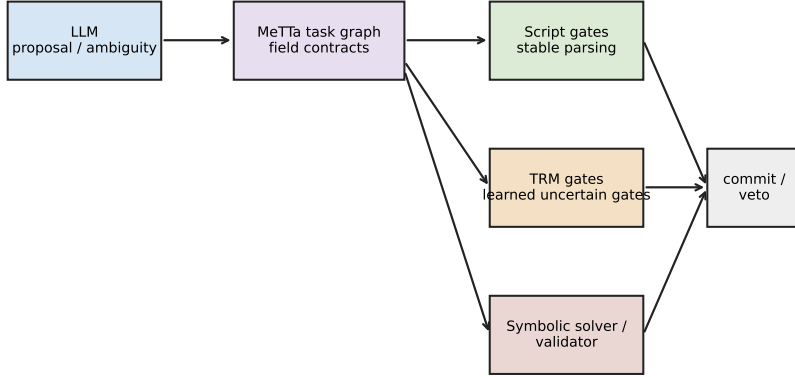
atoms, the LLM can shrink to a proposer; where the task needs latent search or a missing numeric candidate, the circuit still needs a stronger proposal model, teacher candidate, or external solver.

10 Environment-Dependent Compactification

We added two April 28 local Qwen2.5-3B Q4 suites to test whether the compactification claim survives beyond the original repair-rudder rows. The first, `mixed_contract_heldout50`, is a 50-row held-out suite over observable output contracts: summaries with exact word counts, JSON objects, ASCII trees, literal lists, choice labels, and pipe-delimited records. The second, `mixed_contract_hard_ablation30`, shifts the mix toward numeric micro-math, logic labels, computed JSON, state sequences, and deeper trees. Both runs used frozen exact validators and the same local job-cap discipline: 3 GB wrapper cap, 2.6 GB runner child-RSS cap, and structured receipts.

Table 7 gives the main boundary result. On the heldout suite, feedback repair improves exact success from 23/50 to 37/50, with prompt-only MeTTa runtime at 32/50. This is the strongest local evidence that a compact model can serve as a proposal engine when MeTTa/TRM gates own observable output contracts. On the harder suite, the same pattern does not scale cleanly: MeTTa runtime drops to 9/30 and feedback repair reaches only 13/30 versus 12/30 baseline. The blind-repair ablation also reaches 12/30, so the public-validator feedback edge on hard rows is small.

The repair-opportunity subset sharpens the interpretation. In `mixed_contract_hard_ablation30`, `metta_runtime` failed exactly on 21 rows. Blind repair fixed 3 of those rows, while public-validator feedback repair fixed 4. This supports an environment-dependent claim rather than a universal compactification claim: MeTTa/TRM scaffolding is most useful when failure state is verifier-visible and repair is structurally constrained; it is not a substitute for the missing arithmetic, state-transition, or candidate-generation work needed by harder rows.



Principle: route each subtask to the cheapest reliable executor; train TRMs on gates that are neither stable scripts nor safe LLM-only decision

Figure 3: Task-graph allocation schema. MeTTa names the control graph; scripts own stable parsing, TRMs target uncertain verifier-facing gates, symbolic solvers own closure, and the LLM supplies proposals or handles ambiguous surfaces.

11 Task-Graph Allocation

The lesson from the newer camp-gate experiments is that the measurable gain is better described as *task allocation* than as prompt improvement. A MeTTa scaffold is useful because it names the gates in a skill circuit: dimension extraction, anchor extraction, quota extraction, schema commit, symbolic solve, repair, and final commit/veto. Once those gates are explicit, each subtask can be assigned to the cheapest executor that is reliable for that gate. Stable and syntactic gates should usually be scripts. Ambiguous verifier-facing gates are candidates for TRM training. Symbolic solvers and validators should own closure when public constraints are complete. The LLM is then demoted from executor to proposal source and ambiguity resolver.

Figure 3 is the architectural claim in compact form. The MeTTa layer is not merely a natural-language prompt prefix. It is a control-plane representation that makes the skill decomposable. This matters for TRM training because it prevents us from training one opaque “reasoning” target when the data actually contains several distinct targets: field transcription, schema repair, candidate preservation, false-commit avoidance, and symbolic closure acceptance.

The April 29 leakage-safe camp-gate suite tests this idea on a small but diagnostic environment. The rows expose only public puzzle constraints: dimensions, fixed anchors, row/column camp counts, and adjacency rules. The target grid is absent from the prompt. A local Qwen2.5-3B Q4 model cannot solve the raw grid in the earlier run, but when constraints are machine-visible, the public solver closes all 12 rows. The noisy follow-up paraphrases the constraint surface to test extraction rather than grid execution.

Figure 4 shows the allocation boundary. A baseline extraction prompt gives 0/12 repaired solves. A MeTTa schema framing gives 6/12. A graph-gated prompt that separately names dimension, anchor, row-quota, and column-quota gates gives 9/12. A no-model script-owned graph router gives 12/12, because the controlled paraphrases remain parseable by typed extraction gates and the public solver owns final execution. The residual model failures are not failures of camp-grid

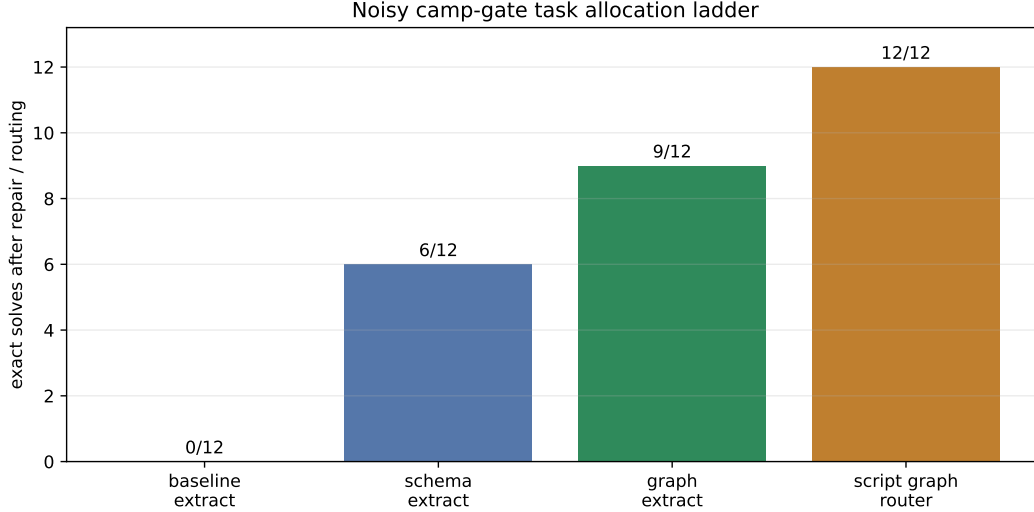


Figure 4: Noisy camp-gate task-allocation ladder. Baseline extraction solves 0/12 after repair; MeTTa schema framing reaches 6/12; explicit graph-gated extraction reaches 9/12; script-owned graph routing reaches 12/12 using only prompt-visible constraints.

reasoning; they are field-transcription errors such as copying column counts into row counts, losing anchors, or shifting coordinates.

This result should be reported with a strict claim boundary. It is not evidence that MeTTa made the 3B model reason like a larger model. It is evidence that MeTTa exposed a better decomposition of the skill. In this decomposition, the LLM becomes optional whenever all public constraints can be captured by stable scripts and a symbolic solver. The TRM training target should move to the gates that scripts do not robustly own: paraphrased field transcription, schema repair accept/reject, and ambiguous commit/veto decisions. This is the concrete compactification mechanism: reducing the LLM footprint by moving deterministic or trainable subgraphs out of the LLM.

12 Interpretation

This supports a staged architecture. First, MeTTa and skill metadata should narrow the valid action space by environment family and verifier label. Second, static MeTTa gates should commit or veto obvious exact/no-gain states. Third, a small LLM or trained commit/veto TRM can choose ambiguous cases. Fourth, post-repair validation should be used as the final commit gate when available.

This distinction matters for the compactification claim. The local 3B can become useful once the symbolic scaffold exposes the right control variables, but the LLM does not become irrelevant everywhere. It becomes less central where the circuit owns validation, repair, extraction, and commit semantics; it remains a weak link where the repair action itself is under-specified or where the candidate must be generated by nontrivial arithmetic, state transition, or search.

13 Publication Scope

This addendum is framed as a methodology and evidence-synthesis paper. It combines historical Hermes TRM baselines, deterministic circuit/projection artifacts, and local small-model benchmark arms. The reporting metrics are target-action accuracy, repair-action accuracy, joint accuracy, false-commit rate, repair-regression rate, and unseen-family transfer. Post-hoc Intellect-3 projection results are reported separately from live benchmark results, and verifier-ceiling results are labeled as training-target definitions rather than trained TRM performance.

14 Conclusion

The current evidence suggests that MeTTa improves the TRM training pipeline most directly by structuring data: it turns semi-failed outputs into typed supervision for validators, repair gates, extraction gates, and commit/veto policies. The local 3B results sharpen the architecture: small LLMs can help as control-plane rudders, but repair-action selection and noisy field transcription need narrower symbolic action spaces, script gates, or trained TRM specialization. The practical conclusion is a compactification method for agent skills: decompose the skill into a typed task graph, assign each gate to the smallest reliable executor, and use the resulting traces to train TRM components that improve the next generation of the skill.

References

- [1] Ben Goertzel. Reflective metagraph rewriting as a foundation for an agi language of thought, 2021. URL <https://arxiv.org/abs/2112.08272>.
- [2] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL <https://arxiv.org/abs/2510.04871>.
- [3] Lucius Gregory Meredith, Ben Goertzel, Jonathan Warrell, and Adam Vandervorst. Meta-metta: An operational semantics for metta, 2023. URL <https://arxiv.org/abs/2305.17218>.
- [4] Prime Intellect Team. INTELLECT-3: A 100b+ moe trained with large-scale rl. <https://www.primeintellect.ai/blog/intellect-3>, 2025. Includes links to the INTELLECT-3 technical report, prime-rl, verifiers, and Environments Hub. Accessed 2026-04-26.
- [5] Sakana AI. Sakana fugu: A multi-agent orchestration system as a foundation model. <https://sakana.ai/fugu-beta/>, April 2026. Beta announcement and technical overview. Accessed 2026-04-30.
- [6] Ryan Teknium, Jeffrey Quesnelle, and Chen Guang. Hermes 3 technical report, 2024. URL <https://arxiv.org/abs/2408.11857>.
- [7] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model, 2025. URL <https://arxiv.org/abs/2506.21734>.
- [8] Xiyang Wu, Zongxia Li, Guangyao Shi, Alexander Duffy, Tyler Marques, Matthew Lyle Olson, Tianyi Zhou, and Dinesh Manocha. Co-evolving llm decision and skill bank agents for long-horizon tasks, 2026. URL <https://arxiv.org/abs/2604.20987>. Version 1, submitted 2026-04-22.